## Project Scenario: University Management System

### Objective

The goal of this project is to create a database for a University Management System that manages information about students, courses, and enrollments. This project will demonstrate the use of primary keys to uniquely identify records and foreign keys to establish relationships between tables.

## Step 1: Database Design

We will create the following tables:

1. **Students**: To store information about students.
2. **Courses**: To store information about courses offered by the university.
3. **Enrollments**: To manage the relationship between students and courses.

## Table Structure

1. **Students Table**
    - `StudentID` (INT, Primary Key, Auto-increment)
    - `FirstName` (NVARCHAR(50), NOT NULL)
    - `LastName` (NVARCHAR(50), NOT NULL)
    - `Email` (NVARCHAR(100), NOT NULL)
2. **Courses Table**
    - `CourseID` (INT, Primary Key, Auto-increment)
    - `CourseName` (NVARCHAR(100), NOT NULL)
    - `Credits` (INT, NOT NULL)
3. **Enrollments Table**
    - `EnrollmentID` (INT, Primary Key, Auto-increment)
    - `StudentID` (INT, Foreign Key referencing Students)
    - `CourseID` (INT, Foreign Key referencing Courses)
    - `EnrollmentDate` (DATE)

## Step 2: SQL Commands

### Create the Database

```
CREATE DATABASE UniversityDB;
GO
```

```
USE UniversityDB;
GO
```

### Create the Tables

#### Create Students Table

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY IDENTITY(1,1),
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100) NOT NULL UNIQUE
);
GO
```

#### Create Courses Table

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY IDENTITY(1,1),
    CourseName NVARCHAR(100) NOT NULL,
    Credits INT NOT NULL CHECK (Credits > 0)
);
GO
```

#### Create Enrollments Table

```
CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY IDENTITY(1,1),
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
GO
```

## Step 3: Insert Sample Data

#### Insert Students

```
INSERT INTO Students (FirstName, LastName, Email)
VALUES
('Alice', 'Johnson', 'alice.johnson@example.com'),
('Bob', 'Smith', 'bob.smith@example.com'),
('Charlie', 'Brown', 'charlie.brown@example.com');
GO
```

#### Insert Courses

```
INSERT INTO Courses (CourseName, Credits)
VALUES
('Database Systems', 3),
('Data Structures', 4),
('Web Development', 3);
GO
```

#### Insert Enrollments

```
INSERT INTO Enrollments (StudentID, CourseID, EnrollmentDate)
VALUES
(1, 1, '2024-09-01'),  -- Alice enrolled in Database Systems
(1, 2, '2024-09-02'),  -- Alice enrolled in Data Structures
(2, 1, '2024-09-01'),  -- Bob enrolled in Database Systems
(3, 3, '2024-09-03');  -- Charlie enrolled in Web Development
GO
```

### Step 4: Querying the Data

#### Select All Students

```
SELECT * FROM Students;
GO
```

#### Select All Courses

```
SELECT * FROM Courses;
GO
```

**Select All Enrollments with Student and Course Information**

```
SELECT E.EnrollmentID, S.FirstName, S.LastName, C.CourseName,
E.EnrollmentDate
FROM Enrollments E
JOIN Students S ON E.StudentID = S.StudentID
JOIN Courses C ON E.CourseID = C.CourseID;
GO
```

## Summary of Keys Used

1. **Primary Key**:
    - Each table has a primary key (`StudentID`, `CourseID`, `EnrollmentID`) to uniquely identify each record.
2. **Foreign Key**:
    - The `Enrollments` table has `StudentID` and `CourseID` as foreign keys, establishing relationships with the `Students` and `Courses` tables, respectively.